

4. Microservices and Event Driven Architectures

Slide 1

Microservices and event-driven architectures change how teams own, connect and operate applications. We will evaluate selective notification extraction in our hypothetical banking pilot, then examine contracts, delivery behaviour and evidence of reliable business effects.

Slide 2

Use the objectives to move from technology labels to decisions that analysts can examine. A service boundary needs a business responsibility and an accountable owner. An interaction needs a clear meaning: requesting work, reporting a fact or retrieving information. Delivery and recovery behaviour determine what customers experience when systems fail or disagree temporarily. Contracts and operational evidence connect those choices to acceptance criteria. In the running case, we will ask whether a separate Notification Service can meet a new team's release needs while protecting the established application workflow. Encourage learners to challenge a proposal with a concrete failure example, rather than counting how many services or events it contains.

Slide 3

Microservices and event-driven architecture address different aspects of a solution. Microservices organise cohesive capabilities into services that can evolve and deploy independently through stable contracts. Event-driven interactions let a producer announce a fact and consumers decide how to respond. A modular application can publish events without being decomposed into many services; a microservice can use request-response when that interaction fits. In our proposed extension, independent notification releases are the reason to examine a service boundary. Event publication is the proposed way to communicate that an application has been recorded. Neither choice removes the need to define failure behaviour, compatible change and the customer's actual completion state.

Sources: <https://martinfowler.com/articles/microservices.html>
<https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/primer.md>

Slide 4

Start with the behaviour and rules that belong together, then test the proposed boundary against change and consistency needs. A bounded context helps clarify what a model means; it does not prescribe exactly one service. Data ownership means that other services use an agreed contract rather than reach into private tables. Separate schemas or tables can enforce ownership on shared database infrastructure, although shared resources still need capacity and failure controls. For the pilot, application receipt and workflow remain a coherent responsibility. Notification formatting and delivery records can have a different owner. Ask whether the proposed team can change, operate and recover its capability without repeatedly changing the application service.

Sources: <https://learn.microsoft.com/en-us/azure/architecture/microservices/model/microservice-boundaries>
<https://microservices.io/patterns/data/database-per-service.html>

Slide 5

This chapter adds one explicit, invented requirement to the existing teaching case: a notification team now needs independent releases. We are evaluating a selective extraction, not assuming that the entire pilot must become microservices. The Loan Application System

continues to own intake, durable records, workflow and customer-visible processing status for existing customers in one pilot country. The existing Credit Decision Service retains assessment, credit policy, approval and bureau checks. A successful API receipt confirms that the application was recorded. It does not confirm email delivery or credit approval. Ask which notification changes need independence, and what evidence would show that a new service boundary actually provides it.

Slide 6

Read the labelled relationships as a focused interaction view, not a complete execution sequence. The application service owns its database and outbox, then makes `ApplicationRecorded` available through the event broker. The proposed Notification Service consumes that fact and owns notification formatting and delivery records in its own store. It does not update the application's tables directly or decide credit outcomes. The existing credit-service contract remains with the application service. Identity and customer-record dependencies are omitted here for clarity and still exist in the wider solution. For each arrow, ask who owns the contract, what information is necessary and what happens when the destination is unavailable or returns an uncertain result.

This view groups API and workflow behaviour under the application team. Its replicas can still share one common release. The proposed Notification Service has a separate release and private notification state. Both may use the managed platform introduced in Chapter 03. The event arrows show the application publishing and the notification capability consuming. The other arrows show data or service dependencies, not a complete execution sequence. Provider delivery and the notification consumer transaction are expanded later.

Slide 7

Use business intent to distinguish these examples. `SendReceiptNotification` requests an action from a responsible receiver; that receiver still applies its validation and acceptance rules. `ApplicationRecorded` reports an established fact and can interest several consumers. `GetApplicationStatus` asks for information without intending to change the application's business state. The name of a broker or protocol does not turn a command into an event. A command can travel asynchronously, and a query can use request-reply messaging. In this proposal, a notification consumer reacts to the recorded fact according to its own contract. Ask learners how the responsibility changes if the application instead explicitly commands a particular notification to be sent.

Sources:

<https://www.enterpriseintegrationpatterns.com/patterns/messaging/CommandMessage.html>

<https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/primer.md>

<https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs>

Slide 8

When a business operation waits for a downstream result before replying, its latency and outcome depend on that interaction. A timeout bounds the wait, but does not always establish whether the remote action happened. A durable asynchronous handoff allows processing to continue later, with explicit progress, expiry and recovery rules. In our pilot, the API receipt follows durable recording while notification delivery proceeds separately. Customers therefore need precise status wording. Request-reply can itself be implemented over messaging, so distinguish the business completion boundary from whether a program thread blocks. Ask which result the caller needs now, which can arrive later, and how an unresolved result will be investigated.

Sources:

<https://www.enterpriseintegrationpatterns.com/patterns/messaging/RequestReply.html>

Slide 9

A work queue distributes deliveries among workers doing the same job; a failed attempt can cause redelivery. Publish-subscribe gives each matching independent subscription its own delivery, and workers can compete within a subscription. A retained stream lets consumers progress and reread records within the configured retention rules. These patterns overlap and products can combine them. For notifications, choose capabilities that support the required recovery and operating model, rather than adding a stream automatically. Ask whether historical replay is required, how long information remains available, and which ordering scope matters. None of these labels promises one external effect, unlimited retention or a single global order across all work.

Sources: <https://learn.microsoft.com/en-us/azure/architecture/patterns/competing-consumers>
<https://learn.microsoft.com/en-us/azure/architecture/patterns/publisher-subscriber>
<https://kafka.apache.org/intro/>

Slide 10

This illustrative contract uses CloudEvents metadata around a minimal business payload. Specversion 1.0 identifies the envelope specification, not the payload schema. Source /loan-applications together with id E-884 identifies this event; preserve that pair across redelivery and replay, even if a broker assigns another transport identifier. Type bank.loan.recorded.v1 expresses the producer-owned event meaning. Subject L-1042 identifies the application concerned, while data carries applicationId L-1042. These example values are not a bank standard. Consumers need to understand the meaning, not just parse the fields. Test compatibility when adding fields or changing interpretation, and include only necessary information. A stable identity supports duplicate recognition; it does not by itself prevent repeated external effects.

Sources: <https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/spec.md>

Slide 11

Follow application L-1042 through the local commit. The application record and its ApplicationRecorded outbox entry are saved together. Only committed outbox work becomes eligible for publication, and successful API receipt follows the durable application commit. Ask learners to place the broker outside that transaction boundary. A relay can publish E-884 successfully and then lose the acknowledgement, so its next attempt may publish the same event again. Preserve the event's source and identifier across those attempts. Monitor pending outbox work and assign recovery ownership. Receipt confirms a recorded application; assessment and notification delivery have their own subsequent outcomes.

Primary sources:

<https://microservices.io/patterns/data/transactional-outbox.html>

Slide 12

For E-884, combine source /loan-applications, event identifier and consumer identity into the deduplication key. In one local transaction, enforce its uniqueness and create a pending notification intent. Acknowledge the broker after commit. Concurrent duplicate deliveries must not create another intent. A dispatcher calls the delivery provider separately. If that call times out, provider acceptance may already have occurred. Marking success before sending

risks a missing notification; sending before recording success creates a duplicate-send risk on retry. Agree provider-supported idempotency or reconciliation, including retention and scope. An intent, provider acceptance and confirmed delivery are distinct states.

Primary sources:

<https://microservices.io/post/microservices/patterns/2020/10/16/idempotent-consumer.html>
<https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>

At-least-once means one or more deliveries under the stated delivery contract, so duplicates are possible. At-most-once delivery avoids redelivery but can lose work after failure. Exactly-once guarantees have a specific processing scope and do not automatically include an external provider. The flow follows notification work through its stages. A duplicate event creates no additional intent. An existing pending intent still requires its normal controlled dispatch and recovery.

Additional source: <https://kafka.apache.org/40/design/design/>

Slide 13

Begin with the business question: which changes must be observed in order for one application? A broker's ordering guarantee has a defined scope, such as a partition, and does not automatically order every downstream effect. For order-sensitive updates, specify application versions, valid transitions and handling of gaps or stale events. Duplicate detection recognises repeated events; it does not decide how two different events should interact. In this pilot, assessment status and notification progress can differ temporarily. Define acceptable delay, monitoring and reconciliation instead of assuming that waiting always repairs divergence. Publishing events does not require the application record to use event sourcing.

Primary sources:

<https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven>

Slide 14

Compare where workflow decisions live. With choreography, Notification Service reacts to ApplicationRecorded using its own rules. An orchestrated workflow has an explicit owner that tracks progress and directs participants through commands. That coordinator also needs durable state and recovery. A saga applies when a business operation spans local transactions and requires defined handling of partial completion. Compensation follows business rules and may itself fail; an email already sent cannot generally be withdrawn. Extracting notification delivery does not itself require a saga. In this case, a notification failure never automatically reverses the recorded application or an existing credit decision. Recover notification work under its own policy.

Primary sources:

<https://microservices.io/patterns/data/saga.html>
<https://learn.microsoft.com/en-us/azure/architecture/patterns/compensating-transaction>

Slide 15

Distinguish temporary failure from an event the consumer cannot process. Use bounded retries for eligible transient conditions. Quarantine unprocessable events with an alert, diagnostic context and a named recovery owner. Before replay, address the cause and check schema compatibility, deduplication retention, completed effects and ordering relative to newer work. Preserve domain identity, including source and E-884, even if the broker assigns

a new transport identifier. SQS redrive provides a concrete example of this distinction and can interweave old and new messages. Reintroduce work at a controlled rate. Stored failed messages still require a resolution decision and evidence that recovery succeeded.

Primary sources:

<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html>

<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-configure-dead-letter-queue-redrive.html>

Slide 16

Trace an application's business outcome across service boundaries. Record enough context to connect L-1042, E-884 and the notification intent without copying sensitive customer payloads into routine telemetry. Follow outbox age, consumer failures, pending intents and unresolved provider outcomes. A successful broker acknowledgement or trace span does not establish email delivery. Agree who may publish, consume and replay each event, and restrict recovery tools accordingly. Producer and consumer teams should test compatible contract changes and maintain an explicit recovery procedure. Ask who investigates an application whose receipt succeeded but whose notification remains uncertain, and which records that person needs to make a justified decision.

Primary sources:

<https://opentelemetry.io/docs/specs/semconv/messaging/messaging-spans/>

<https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/spec.md>

<https://www.asyncapi.com/docs/reference/specification/v3.0.0>

Slide 17

ADR-004 responds to a new hypothetical need: the notification team wants independent releases. The proposal gives Notification Service ownership of formatting and delivery records behind an event contract. Intake and workflow remain together, while the existing Credit Decision Service retains assessment, approval and bureau checks. Assess whether the proposed boundary enables a notification change without a coordinated application release. Also account for delayed notification progress, extra monitoring and recovery work. Approval of the proposal should depend on evidence for durable publication, local idempotency, provider uncertainty handling and operational ownership. Reconsider the extraction if the release benefit fails to justify those responsibilities.

Primary sources:

<https://martinfowler.com/articles/microservices.html>

<https://learn.microsoft.com/en-us/azure/architecture/microservices/model/microservice-boundaries>

Slide 18

Allow exactly 12 minutes: two to understand, five to design, three to choose evidence and two to report. Event E-884 from /loan-applications, concerning L-1042, arrives twice. The delivery provider times out after a send request, leaving its outcome unknown. Ask groups to identify the local transaction boundary and stable event key, then separate what is known locally from what is known at the provider. Require a justified query or retry policy, controlled replay, recovery ownership and truthful statuses. The recorded application remains valid. Pending assessment continues until the existing credit service supplies its outcome. Notification uncertainty does not justify a new application or a changed credit decision.

Primary sources:

<https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>

<https://microservices.io/post/microservices/patterns/2020/10/16/idempotent-consumer.html>

Slide 19

Commit the unique source, event and consumer key with one notification intent. Test a crash after that commit but before broker acknowledgement: redelivery should retain one intent. Then simulate provider acceptance followed by a lost response. Query or retry only under the provider's documented contract; if it offers no suitable support, apply an explicit operator policy for the uncertain outcome. Controlled replay preserves domain identity and checks completed effects. Receipt remains evidence of durable recording, with Pending assessment while credit assessment is incomplete. Notification state stays separate. Require evidence for both crash windows and distinguish provider acceptance from actual delivery.

Primary sources:

<https://microservices.io/post/microservices/patterns/2020/10/16/idempotent-consumer.html>

<https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>

Slide 20

Use these primary references to check the reasoning behind service boundaries and message contracts. Lewis and Fowler discuss independently deployable services; Microsoft's guidance helps evaluate boundaries. CloudEvents describes event context, while AsyncAPI documents message-driven interfaces. Richardson explains reliable publication and duplicate handling. Topic-specific notes add guidance for provider uncertainty, recovery and observability. All banking names, identifiers and scenarios in this chapter are hypothetical teaching examples. The proposed extraction needs local requirements, service contracts and validation evidence before implementation. Encourage learners to revisit the relevant source when a design decision changes, then record the assumption, accountable owner and test that support the decision.

Primary sources:

<https://martinfowler.com/articles/microservices.html>

<https://learn.microsoft.com/en-us/azure/architecture/microservices/model/microservice-boundaries>

<https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/spec.md>

<https://www.asyncapi.com/docs/reference/specification/v3.0.0>

<https://microservices.io/patterns/data/transactional-outbox.html>

<https://microservices.io/post/microservices/patterns/2020/10/16/idempotent-consumer.html>